Problem Set 7: Shrinkage Methods

```r
if (!require(glmnet)) install.packages("glmnet")
library(glmnet)

# Fresh clean data
data(Hitters)
Hitters = na.omit(Hitters)

# 1. Split: 100 test, rest training
set.seed(42)
test_idx  = sample(1:nrow(Hitters), 100)
train7    = Hitters[-test_idx, ]
test7     = Hitters[test_idx, ]

X_train = model.matrix(Salary ~ ., train7)[, -1]
y_train = train7$Salary
X_test  = model.matrix(Salary ~ ., test7)[, -1]
y_test  = test7$Salary

cat("Train size:", nrow(train7), "| Test size:", nrow(test7), "\n")
```

```
## Train size: 163 | Test size: 100
```

2. Least Squares

```r
ols_fit   = lm(Salary ~ ., data = train7)
ols_train = predict(ols_fit, train7)
ols_test  = predict(ols_fit, test7)

ols_train_mse = mean((y_train - ols_train)^2)
ols_test_mse  = mean((y_test  - ols_test)^2)

cat("OLS Train MSE:", round(ols_train_mse, 2), "\n")
```

```
## OLS Train MSE: 91923.29
```

```r
cat("OLS Test MSE: ", round(ols_test_mse,  2), "\n")
```

```
## OLS Test MSE:  124714
```

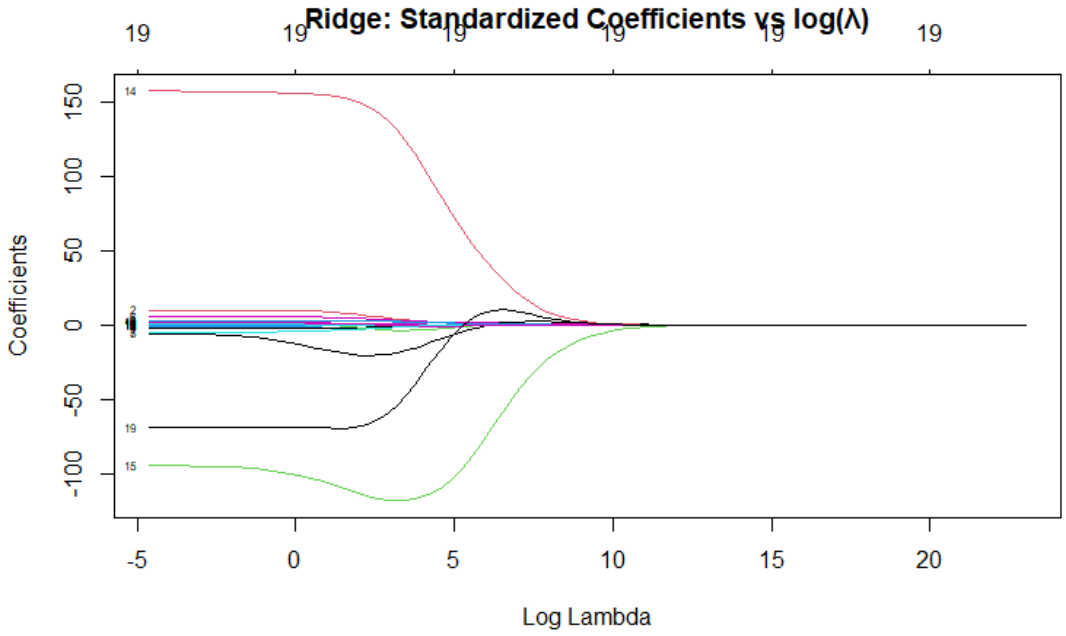
3. Ridge Regression

```r
# Grid of lambda values
lambda_grid = 10^seq(10, -2, length = 100)

ridge_fit = glmnet(X_train, y_train, alpha = 0, lambda = lambda_grid)

# (i) Dimension of coefficient matrix
cat("Coefficient matrix dimensions:", dim(coef(ridge_fit)), "\n")
```

```
## Coefficient matrix dimensions: 20 100

# (ii) L2 norm at 50th and 60th lambda
l2_50 = sqrt(sum(coef(ridge_fit)[-1, 50]^2))
l2_60 = sqrt(sum(coef(ridge_fit)[-1, 60]^2))
cat("L2 norm at lambda[50] =", round(lambda_grid[50], 4), ":", round(l2_50,
4), "\n")

## L2 norm at lambda[50] = 11497.57 : 7.6918

cat("L2 norm at lambda[60] =", round(lambda_grid[60], 4), ":", round(l2_60,
4), "\n")

## L2 norm at lambda[60] = 705.4802 : 65.8542

cat("Larger lambda => smaller L2 norm (more shrinkage)\n")

## Larger lambda => smaller L2 norm (more shrinkage)

# (iv) Test MSE for 50th and 60th lambda
pred_50 = predict(ridge_fit, s = lambda_grid[50], newx = X_test)
pred_60 = predict(ridge_fit, s = lambda_grid[60], newx = X_test)
cat("Test MSE at lambda[50]:", round(mean((y_test - pred_50)^2), 2), "\n")

## Test MSE at lambda[50]: 117860.6

cat("Test MSE at lambda[60]:", round(mean((y_test - pred_60)^2), 2), "\n")

## Test MSE at lambda[60]: 85167.17
```

4. Ridge Coefficient Path Plot (vs λ)

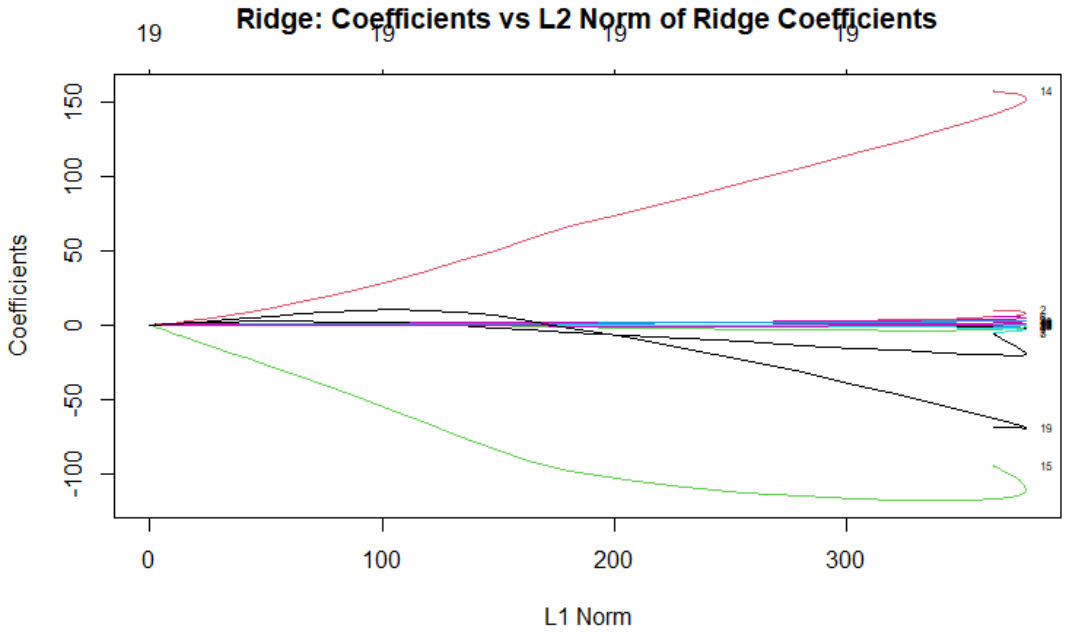
```
plot(ridge_fit, xvar = "lambda", label = TRUE,
     main = "Ridge: Standardized Coefficients vs log(λ)")
```



Interpretation: As λ increases, all coefficients are progressively pulled closer to zero. However, none of them become exactly zero—ridge regression keeps all predictors in the model, just with smaller coefficients.

5. Coefficient Path vs L2 Norm Ratio

```
# Plot using xvar = "norm" which shows L2 norm of ridge coef
plot(ridge_fit, xvar = "norm", label = TRUE,
     main = "Ridge: Coefficients vs L2 Norm of Ridge Coefficients")
```



Interpretation: Moving to the left (i.e., a smaller L2 norm, which corresponds to a larger λ), all coefficients get closer to zero. This represents the same relationship as the λ plot, but with the horizontal axis flipped.

6. Ridge with Cross-Validation

```
set.seed(42)
cv_ridge = cv.glmnet(X_train, y_train, alpha = 0)
best_lambda_ridge = cv_ridge$lambda.min
cat("Best lambda (CV):", round(best_lambda_ridge, 4), "\n")

## Best lambda (CV): 29.8318

ridge_pred_cv = predict(cv_ridge, s = best_lambda_ridge, newx = X_test)
ridge_test_mse = mean((y_test - ridge_pred_cv)^2)
cat("Ridge Test MSE (CV lambda):", round(ridge_test_mse, 2), "\n")

## Ridge Test MSE (CV lambda): 104451.5
```

7. LASSO with Cross-Validation

```
set.seed(42)
cv_lasso = cv.glmnet(X_train, y_train, alpha = 1)
best_lambda_lasso = cv_lasso$lambda.min
cat("Best lambda (CV):", round(best_lambda_lasso, 4), "\n")

## Best lambda (CV): 2.8476

lasso_pred = predict(cv_lasso, s = best_lambda_lasso, newx = X_test)
lasso_test_mse = mean((y_test - lasso_pred)^2)
```

```r
lasso_coefs = coef(cv_lasso, s = best_lambda_lasso)
nonzero     = sum(lasso_coefs != 0) - 1  # exclude intercept
cat("LASSO Test MSE:", round(lasso_test_mse, 2), "\n")

## LASSO Test MSE: 116353.6

cat("Number of non-zero coefficients:", nonzero, "\n")

## Number of non-zero coefficients: 17
```

8. Elastic Net with Cross-Validation

```r
# Try several alpha values and pick the best via CV
alphas    = seq(0.1, 0.9, by = 0.1)
cv_errors = sapply(alphas, function(a) {
  cv_fit = cv.glmnet(X_train, y_train, alpha = a)
  min(cv_fit$cvm)
})

best_alpha = alphas[which.min(cv_errors)]
cat("Best alpha:", best_alpha, "\n")

## Best alpha: 0.7

set.seed(42)
cv_enet     = cv.glmnet(X_train, y_train, alpha = best_alpha)
best_lam_en = cv_enet$lambda.min
cat("Best lambda (CV):", round(best_lam_en, 4), "\n")

## Best lambda (CV): 3.3773

enet_pred    = predict(cv_enet, s = best_lam_en, newx = X_test)
enet_test_mse = mean((y_test - enet_pred)^2)

enet_coefs   = coef(cv_enet, s = best_lam_en)
nonzero_en   = sum(enet_coefs != 0) - 1
cat("Elastic Net Test MSE:", round(enet_test_mse, 2), "\n")

## Elastic Net Test MSE: 116299

cat("Non-zero predictors:", nonzero_en, "\n")

## Non-zero predictors: 17

cat("Non-zero predictor names:\n")

## Non-zero predictor names:

print(rownames(enet_coefs)[enet_coefs[,1] != 0 & rownames(enet_coefs) !=
"(Intercept)"])

##  [1] "AtBat"      "Hits"       "HmRun"      "RBI"        "Walks"
##  [6] "Years"      "CHits"      "CHmRun"     "CRuns"      "CRBI"
```

```
## [11] "CWalks"     "LeagueN"    "DivisionW"  "PutOuts"    "Assists"
## [16] "Errors"     "NewLeagueN"
```

9. Final Comparison

```
comparison = data.frame(
  Method     = c("OLS", "Ridge (CV)", "LASSO (CV)", "Elastic Net (CV)"),
  Test_MSE   = round(c(ols_test_mse, ridge_test_mse, lasso_test_mse,
enet_test_mse), 2)
)
print(comparison)
```

```
##             Method Test_MSE
## 1              OLS 124714.0
## 2       Ridge (CV) 104451.5
## 3       LASSO (CV) 116353.6
## 4 Elastic Net (CV) 116299.0
```

Interpretation: Shrinkage techniques such as Ridge, LASSO, and Elastic Net often perform better than OLS on test data because they help control overfitting. LASSO has the added advantage of setting some coefficients exactly to zero, which effectively performs variable selection and improves interpretability. Elastic Net blends the penalties of Ridge and LASSO, and in some cases can outperform both. Ultimately, the model with the lowest test MSE provides the most accurate salary predictions.
